

REPORT

Title

Implementation of Breadth First Search (BFS) Algorithm Using Python

Aim

To implement the Breadth First Search (BFS) algorithm in Python for graph traversal.

Objective

To traverse all vertices of a graph level by level using a queue.

Requirements

- Python 3.x

Theory

Breadth First Search (BFS) is a graph traversal algorithm that visits vertices level by level. It starts from a source vertex, visits all its immediate neighbors, and then proceeds to the next level. BFS uses a queue to maintain the order of traversal. It guarantees the shortest path in terms of the number of edges in an unweighted graph.

Algorithm

1. Create an empty queue and a visited set.
2. Add the starting vertex to both the queue and visited set.
3. While the queue is not empty:
 - Remove the front vertex from the queue.
 - Display the vertex.
 - Visit all unvisited adjacent vertices.
 - Mark them as visited and add them to the queue.

4. Stop when the queue becomes empty.

Program

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    visited.add(start)

    print("BFS Traversal:")

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

bfs(graph, 'A')
```

Sample Output

```
BFS Traversal:
A B C D E F
```

Result

The Breadth First Search (BFS) algorithm was successfully implemented in Python and traversed the graph in level-order starting from the source vertex **A**.

Conclusion

The BFS algorithm efficiently traverses all reachable vertices of a graph using a queue. It is widely used for shortest path finding in unweighted graphs, network routing, web crawling, and graph analysis.